CSC202 Spring 2026

Assignment 4

You will write code to produce a concordance (similar to the index of a book) for a large text file. This code will rely on a hash table implementation, which you will use both as a *set* and as a *dictionary*.

Create Git Repository

Create a private GitHub repository where all group members are collaborators. You must make sure no one outside your group sees your code.

The repository should contain a **.gitignore** (nothing new here) and **main.py**:

main.py

```
from typing import *
from dataclasses import dataclass
import unittest
import sys
import string
sys.setrecursionlimit(10**6)

class Tests(unittest.TestCase):
    pass

if (__name__ == '__main__'):
    unittest.main()
```

Grading

Grading will work as in previous assignments: you will submit your files (not just **main.py** this time) to Gradescope, where automatic tests will run against the code you've given in **main.py**. Some of Gradescope's test results will be invisible prior to the deadline. The final grade will be based on the auto-tests and on an "eyeball score" assessing your code's quality.

Code Requirements

Do not use Python's built-in dictionary and set types! The idea here is to achieve what those things achieve using your own hash table implementation.

Provide test cases for all functions except where told otherwise.

Use `@dataclass(frozen=True)` on all classes except where told otherwise.

What is a *Concordance*?

A concordance is an alphabetical list of words that occur in some large document, each word displayed along with a list of the places in the document where that word occurs. (This is very similar to the index at the back of a book, where you see an alphabetical list of key words along with the locations—page numbers—where each key word appears.)

The goal in this assignment is to process a document (text file) to generate a concordance with line numbers for each word.

At an abstract level, we're dealing with a *map/dictionary*: a mapping from word-in-document to the line numbers where said word occurs. The *key* is a word from the document; the associated *value* is a collection of line numbers.

A hash table is ideal for map/dictionary situations, and it's how you'll store a concordance

Since there are certain words like "a", "the", and "of" that are very frequent and which users are unlikely to want to track, there will be an extra input text file containing words to ignore: *stop-words*[1]. These are words to leave out of the concordance even if they do appear in the document.

Removing Punctuation

For each line of text in the document you're making a concordance for, do the following:

- Remove all occurrences of the apostrophe character (‘), e.g., the word "don't" would simply become "dont". The `replace()` method of a string can help.

[1] The term *stop word* is misleading, it seems to indicate a word that causes processing to stop. *Stop word* is an old term of art in natural language processing, and we're stuck with it now.

- Convert all characters in `string.punctuation` (a reference string stored in the `string` module) to spaces. Again, the `replace()` method can help—but to be clear, it's not possible to use a *single* `replace()` call to do this.
- make all characters lower-case, using the `lower()` function
- Split the string into tokens (substrings) using the `split()` method of a string.
- Each token that returns `True` when its `isalpha()` method is called should be considered a *word*. All other tokens should be ignored. (You'll also, of course, ignore words that are named in your stop-words file.)

Sample Inputs and Output

Sample Text File

```
This is a sample data ((text)) file, to be
processed by your word-concordance program!!!

A REAL data file is MUCH bigger. Gr8!
```

Sample Stop-Words File:

```
a
about
be
by
can
do
i
in
is
it
of
on
the
this
to
was
```

Sample Result File:

```
bigger: 4
concordance: 2
data: 1 4
file: 1 4
much: 4
processed: 2
program: 2
```

```
real: 4
sample: 1
text: 1
word: 2
your: 2
```

Note the following:

1. There is no distinction made between upper- and lower-case letters (“CaT” is the same word as “cat”).
2. Blank lines are counted in line numbering.

General Algorithm

1) Read the stop-words file and store all its words using your implementation of a hash table. Remember, you’ll be using the same hash table implementation for storing stop words that you use for storing the concordance itself. However, in the case of stop words, you’ll be using the hash table as a *set*, whereas with the concordance itself you’ll be using the hash table as a *map/dictionary*. (When it comes to stop words, you won’t actually need to store line numbers; just supply a default value).
2) The concordance itself will be in a separate hash table from the stop words hash table (though both use the same hash table *implementation*). Process the input file one line at a time to build the concordance. The concordance-storing hash table should only contain non-stop words as keys (use the stop words hash table to filter out the stop words that you encounter in the input file). Associated with each *key* is its *value*: a linked list containing the line numbers where the key appears. Do not include a given line number more than once; there should be no duplicates in the linked list of line numbers.
3) Generate a text file containing the concordance words printed out in alphabetical order along with their line numbers. Each line of the output starts with a word followed by a colon, then the line numbers separated by single spaces:

   data: 1 4

   There is no space after the last line number. Make sure to match the sample output file.

Hash Function

The *hash function* should take as input a string `txt` containing one or more characters and return an integer. Here is the hash function you should use:

∑_(i=0 to n-1) ord(txt[i]) * 31^(n-1-i)

where n = len(txt).

To make this hash function more efficient, don't compute a whole new power of 31 at every cycle of the loop (you will probably be using a for- or a while-loop). Instead, design this function as a loop where your pow-of-31 variable gets multiplied by 31 with each increase of 'n'. This is a technically a specific application of Horner's method.

As usual, you'll need the modulo operation to ensure that the *hash codes* that result from this hash function get turned into usable bin indices in your hash table.

Separate Chaining

Your hash table should use *separate chaining*, meaning that each of its bins can contain multiple key-value pairs. Remember that for a concordance, a key is a non-stop word and the value is a linked list of unique line numbers where that word occurs in the document. A bin itself is going to be a linked list of key-value pairs. Note that linked lists feature on multiple levels in this assignment.

Maximum Load Factor

Your hash table should be able to grow. Specifically, its maximum load factor should be 1.0.

After insertion of an item, if the number of key-value pairs is greater than or equal to the number of bins, you should grow the hash table size.

The default starting size of your hash table should be 128 bins. When increases are necessary, double the size of the table. When you do this, "re-hash" all the key-value pairs in the table to find their new locations in the expanded table.

Python Built-In Data Structures

As stated earlier, do not use Python's built-in dictionary or set for this assignment. These both rely on Python's implementation of a hash table, and the idea in this assignment is to rely on *your* implementation of a hash table.

You will of course be using Python's array type (`List`) in multiple places, e.g., to represent the hash table itself, plus as the return type of several functions defined below.

Note, though, that you'll be using a *linked list* to represent each of the hash table's bins, as well as to represent the line numbers associated with each key.

Data Definitions

Include the following:

- `IntList`: a linked list of integers, used to store the unique line numbers associated with a word in the concordance.
- `WordLines`: a non-frozen (mutable) dataclass representing a key-value pair. This contains (a) the key—a word—and (b) the meant-to-be-mutated value associated with that word: an `IntList` representing the unique line numbers where that word occurs.
- `WordLinesList`: a linked list of `WordLines`'s. (This is one of the bins in your hash table).
- `HashTable`: a non-frozen (mutable) dataclass containing (a) an array (`List`) of `WordLinesList`'s, and (b) a count of how many `WordLines`'s (key-value pairs) are stored in the hash table.

Note that we have to make `WordLines` and `HashTable` mutable (no `frozen=True`). However, the only things that your code should actually mutate are the two attributes of `HashTable` and the first attribute of `WordLines`.

Functions

Fill in the bodies of these functions (feel free to keep the given headers and purpose statements):

```
# Return the hash code of 's' (see assignment description).
def hash_fn(s: str) -> int:
  pass

# Make a fresh hash table with the given number of bins 'size',
# containing no elements.
def make_hash(size: int) -> HashTable:
  pass

# Return the number of bins in 'ht'.
def hash_size(ht: HashTable) -> int:
  pass

# Return the number of elements (key-value pairs) in 'ht'.
def hash_count(ht: HashTable) -> int:
  pass

# Return whether 'ht' contains a mapping for the given 'word'.
```

```
def has_key(ht: HashTable, word: str) -> bool:
  pass

# Return the line numbers associated with the key 'word' in 'ht'.
# The returned list should not contain duplicates, but need not be sorted.
def lookup(ht: HashTable, word: str) -> List[int]:
  pass

# Record in 'ht' that 'word' has an occurrence on line 'line'.
def add(ht: HashTable, word: str, line: int) -> None:
  pass

# Return the words that have mappings in 'ht'.
# The returned list should not contain duplicates, but need not be sorted.
def hash_keys(ht: HashTable) -> List[str]:
  pass

# Given a hash table 'stop_words' containing stop words as keys, plus
# a sequence of strings 'lines' representing the lines of a document,
# return a hash table representing a concordance of that document.
def make_concordance(stop_words: HashTable, lines: List[str]) -> HashTable:
  pass

# Given an input file path, a stop-words file path, and an output file path,
# overwrite the indicated output file with a sorted concordance of the input
file.
def full_concordance(in_file: str, stop_words_file: str, out_file: str) -> None:
  pass
```

Only a few of these functions should require traversing the entire hash table: `hash_keys`, `make_concordance`, `full_concordance`, and `add` (on the occasion when a resize is required).

All of these should be defined in the file main.py, in such a way that Gradescope tests can say

```
from main import make_hash, hash_size, hash_count, has_key,
lookup, add, hash_keys, make_concordance, full_concordance
```

Writeup

Write a short description of running your code on a large text file (> 1 megabyte), including the total length of the output file, plus a hand-picked random chunk of five consecutive lines from

the output file. Choose one of those five lines and verify by hand that the given word does in fact occur in the specified locations within the large input text file.

You'll need to come up with a stop-words file to use for this experiment—feel free to create one manually. You'll quickly get a sense for what kinds of words should be treated as stop words.

To find a text file larger than a megabyte, consider books, *long* books, especially ones whose text is in the public domain and thus findable online.

This writeup should be in the form of a PDF with the name **writeup.pdf**, submitted to Gradescope along with the rest of your files.

For those of you who submit on Gradescope using the GitHub-repository option, you will need to `git add` *your* ***writeup.pdf*** *to your repository in order for the writeup to be submitted on Gradescope.*

Turning in on Gradescope

You should turn in **main.py**, **writeup.pdf**, plus the large text file and the stop-words file that you describe using in your writeup.

Hand in your files using Gradescope. Whoever submits, be sure to designate all of your team members using the Group Members button.

Your **main.py** file should not generate any output files or require any input files to be present *when loaded as a module*. Do not worry about code in your **main.py** file that is guarded by `if (__name__ == "__main__")`. Such code will not run when **main.py** is loaded as a module. Thus, it is okay for you to have test code in **main.py** that creates/loads files, just as long as that test code only gets called inside the aforementioned if-statement.